



Интеграција модула, уговори и подаци

EquipmentReservation као основа за интеграцију и даљи развој

Од ове вежбе прати се исти извршиви пример

EquipmentReservation није изолован snippet, већ solution који се надограђује кроз више тема.

Студент прати како се исти систем развија кроз интеграцију, AI workflow, MCP и guardrails.

КЉУЧНА ПОРУКА

Главна улазна тачка је EquipmentReservation.sln.

Отварање и провера наставног примера

Прва провера је да solution може да се restore-ује, build-ује и тестира.



```
1 cd examples/ers-ai-workflow
2 dotnet restore EquipmentReservation.sln
3 dotnet build EquipmentReservation.sln -c Release --no-restore
4 dotnet test EquipmentReservation.sln -c Release --no-build
```

Шта је важно

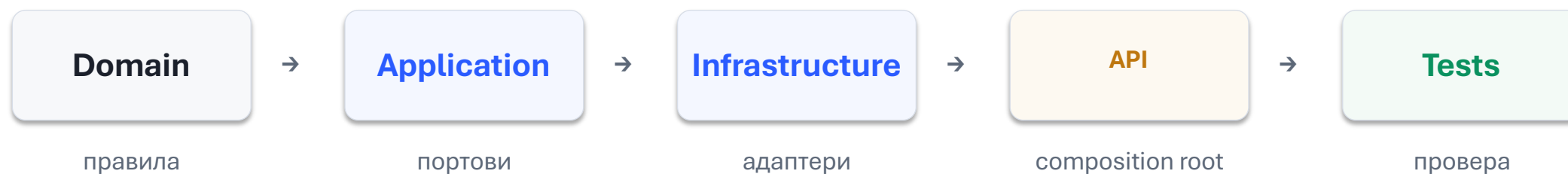
Радимо над целим solution-ом.

Команде су део доказа, не декоративан додатак.

Пад build-а или теста мора се решити пре даље интеграције.

Један solution, јасне границе

MCP и Guardrails су у истом solution-у, али не постају зависности пословног језгра.



КЉУЧНА ПОРУКА

Dependency Rule је важнији од физичког распореда фолдера.

Структура solution-a

Сваки пројекат има другачију одговорност.

Пројекат	Одговорност
Domain	ентитети и пословна правила; без пројектних зависности
Application	use-case-ови и портови према другим модулима
Infrastructure	имплементације портова
Api	composition root и HTTP граница
Mcp	контролисано излагање контекста AI клијенту
Guardrails	детерминистичке политике за ризичне позиве
Tests	NUnit провере домена, use-case-a и guardrail-a

Пословно правило остаје у Domain слоју

InventoryItem не зна за HTTP, базу, MCP нити AI.

Он штити invariant: количина мора бити позитивна и не може се резервисати више јединица него што је расположиво.

КЉУЧНА ПОРУКА

Исто правило користе API, тестови и будући adapter-и без дуплирања.

Валидација захтева је одвојена од модела

Validator проверава облик улаза пре покретања пословног тока.

```
1 public static void ValidateAndThrow(  
2     CreateReservationCommand command)  
3 {  
4     ArgumentException.ThrowIfNull(command);  
5     if (command.RequestId == Guid.Empty)  
6         throw new ArgumentException(  
7             "Request id is required.");  
8     if (command.Quantity <= 0)  
9         throw new ArgumentOutOfRangeException(  
10             nameof(command));  
11 }
```

Шта је важно

Модел само носи стање.

Validator проверава обавезна поља и опсег.

Handler покреће проверу пре позива портова.

Уговор између Reservations и Inventory је експлицитан

Application описује шта use-case заиста треба од Inventory дела.

Не прослеђује ORM ентитет, не отвара интерно складиште и не крши границу другог модула.

КЉУЧНА ПОРУКА

Ово је практична примена ISP и DIP.

Application дефинише порт

Use-case зависи од уговора који му је потребан.

```
1 public sealed record ReserveInventoryRequest(  
2     Guid EquipmentId, int Quantity, Guid ReservationId);  
3  
4 public sealed record ReserveInventoryResult(  
5     bool Success, string? ErrorCode);  
6  
7 public interface IInventoryModule  
8 {  
9     Task<ReserveInventoryResult> ReserveAsync(  
10         ReserveInventoryRequest request,  
11         CancellationToken cancellationToken);  
12 }
```

Шта је важно

Порт је у Application слоју.

Infrastructure реализује уговор.

Use-case не зна интерне детаље Inventory модула.

Handler води пословни ток

Handler валидира улаз, проверава поновљени захтев и позива Inventory кроз порт.

```
1 CreateReservationCommandValidator
2   .ValidateAndThrow(command);
3 await using var lease = await _requestLock
4   .AcquireAsync(command.RequestId, ct);
5 var existing = await _reservations
6   .FindByRequestIdAsync(command.RequestId, ct);
7 if (existing is not null)
8   return Map(existing, replayed: true);
9 var inventoryResult = await _inventory.ReserveAsync(
10   new(command.EquipmentId, command.Quantity,
11     reservationId), ct);
12 var status = inventoryResult.Success
13   ? ReservationStatus.Confirmed
14   : ReservationStatus.Rejected;
15 var reservation = new Reservation(
16   reservationId, command.RequestId,
17   command.EquipmentId, command.StudentId,
18   command.Quantity, status,
19   inventoryResult.ErrorCode);
20 await _reservations.AddAsync(reservation, ct);
21 return Map(reservation, replayed: false);
```

Шта је важно

Модел нема пословне методе.

Inventory правило остаје иза порта.

Application враћа стабилан исход API слоју.

Idempotency key

Поновљени исти захтев не сме поново умањити залиху.

RequestId

један захтев, један ефекат

Ако исти захтев стигне поново, постојећа резервација се враћа без другог позива на промену стања.

Идемпотентност мора бити закључана тестом

Коментар није доказ да понашање заиста постоји.

```
1 var first = await handler.HandleAsync(command, ct);
2 var replay = await handler.HandleAsync(command, ct);
3 var available = await inventory.GetAvailableAsync(
4     command.EquipmentId, ct);
5
6 Assert.That(first.Outcome,
7     Is.EqualTo(CreateReservationOutcome.Confirmed));
8 Assert.That(replay.ReservationId,
9     Is.EqualTo(first.ReservationId));
10 Assert.That(replay.Replayed, Is.True);
11 Assert.That(available, Is.EqualTo(3));
```

Шта је важно

Тест проверава уговор између модула.

Поновљени позив не сме да направи нови side effect.

Проверавају се и исход и преостала количина.

Интеграција се брани доказима

Није довољно рећи да су слојеви раздвојени.

01

Build

целокупан solution се компајлира

02

Tests

пословни и интеграциони сценарији пролазе

03

Diff

промена је ограничена на очекиване слојеве

04

Contract

уговор између модула је стабилан

API је composition root

HTTP граница не сме постати место пословних правила.

API РАДИ

прима HTTP захтев

мапира DTO у команду

позива handler

мапира Result у одговор

API НЕ РАДИ

не проверава залиху директно

не мења Domain стање мимо use-case-a

не чита интерну базу другог модула

не садржи пословну одлуку

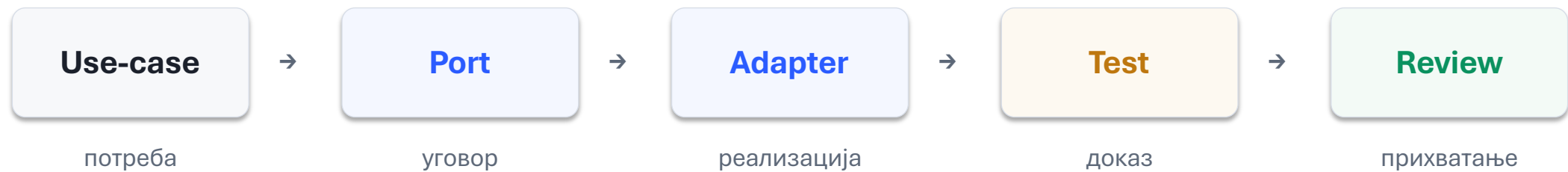
Интеграциони ризици

Сваки ризик треба да има начин провере.

Ризик	Провера
Погрешан смер зависности	project references и build
Дуплирана правила у API-ју	code review и diff
Поновљен захтев прави нови ефекат	idempotency тест
Порт пропушта интерне детаље	преглед Application уговора
MCP/Guardrails улазе у Domain	dependency rule провера

Од модула до заједничког производа

Интеграција је договор о граници, а не само позив методе.



КЉУЧНА ПОРУКА

Модули се повезују преко стабилних уговора који имају пословно значење.

Јасна граница олакшава AI workflow

Када су границе стабилне, AI предлог се лакше проверава.

01

Контекст

погођени пројекти су
познати

02

Ограничења

зна се шта не сме да се
мења

03

Провера

build, test и diff су довољни
сигнали

Рад на вежби

Циљ је да студенти користе исти пример као извршиви систем.

- 01 Отворити EquipmentReservation.sln.
- 02 Проверити build и test команде.
- 03 Пронаћи Domain правило и његов Application порт.
- 04 Објаснити како CreateReservationHandler користи Inventory.
- 05 Показати тест који штити идемпотењнтност или уговор између модула.

Суштина Вежбе 5

Интеграција није супротност Clean Architecture-у.

**Добра интеграција значи да модули сарађују преко јасних уговора,
док пословно језгро остаје заштићено од детаља транспорта,
складишта и развојних алата.**

КЉУЧНА ПОРУКА

То припрема терен за контролисан AI рад.

За наредни термин

Припремити solution за AI workflow.

- 01 Забележити како се solution покреће и тестира.
- 02 Описати границе Domain, Application, Infrastructure и API пројеката.
- 03 Изабрати један мали захтев за промену правила.
- 04 Навести фајлове које промена сме да погађа.
- 05 Припремити проверу build/test/diff за следећу вежбу.